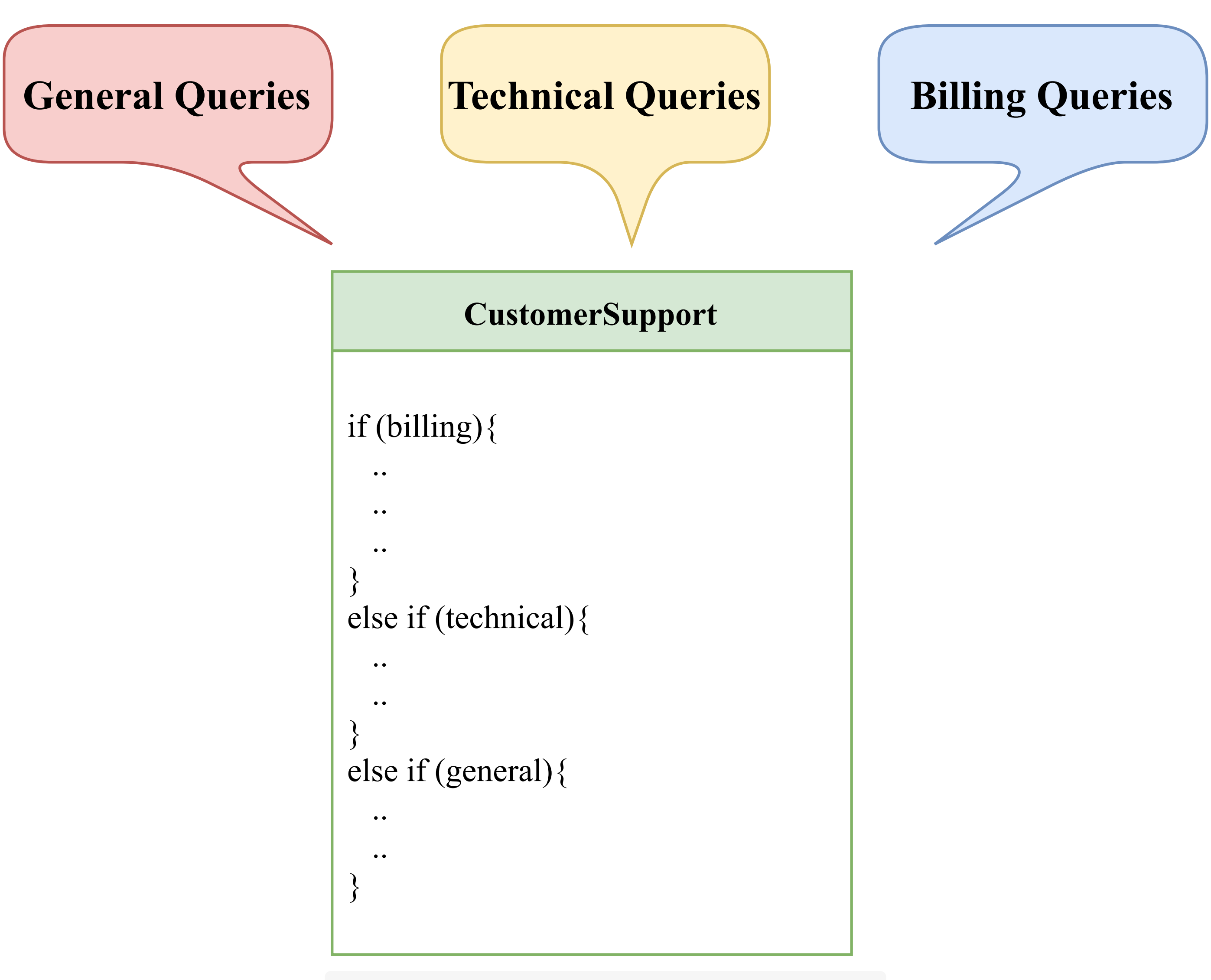
Chain of Responsibility Pattern

The Chain of Responsibility design pattern allows an object to pass a request down a chain of handlers. The originating object sends the request to a chain of handlers so that each handler in the chain can process it rather than sending it directly to a specific handler.

Imagine a company where all customer queries (technical, billing, general information) are managed by a single class, `CustomerSupport`. This class contains complex conditional logic to handle different types of queries. This leads to high complexity, inflexibility of the system, and violation of the single responsibility principle.

The image below illustrates the situation very accurately:



Chain of Responsibility-Customer Support Example

This issue can be solved using the Chain of Responsibility pattern. It addresses such issues by creating a chain of multiple handlers, each of which is responsible for a specific type of query. A query is passed along the chain until it reaches the required handler capable of processing it.

A query arrives at the first handler, say, `TechnicalSupportHandler`. If it's a technical issue, this handler processes it. If not, it passes the query to the next handler in the chain, say, `BillingHandler`, and so on. This process continues until the query is handled or reaches the end of the chain.

The proposed approach aims to simplify the customer support system, making it more flexible, maintainable, and scalable.

Real-World Example

The process flow in a restaurant can be compared to the Chain of Responsibility pattern in real life. As soon as you walk in, the host or hostess attends you and makes seating arrangements. If you have a specific request, such as a preferred table, they will determine whether it can be accommodated or if you will have to wait. After that, a waiter takes over to oversee your meal service and take orders. The chef becomes involved when there are particular demands, such as a customized dish. Finally, if there is an issue that requires higher authority, such as a billing discrepancy, the manager steps in.

This process of managing requests at various levels, with each employee in charge of particular duties, is similar to the Chain of Responsibility pattern in which a request moves through a series of handlers before being processed.

Structure of Chain of Responsibility Pattern

The Chain of Responsibility pattern has some key components:

- **Handler**:

  An abstract class or interface that defines how a request can be handled and holds the reference to the next handler in the chain.

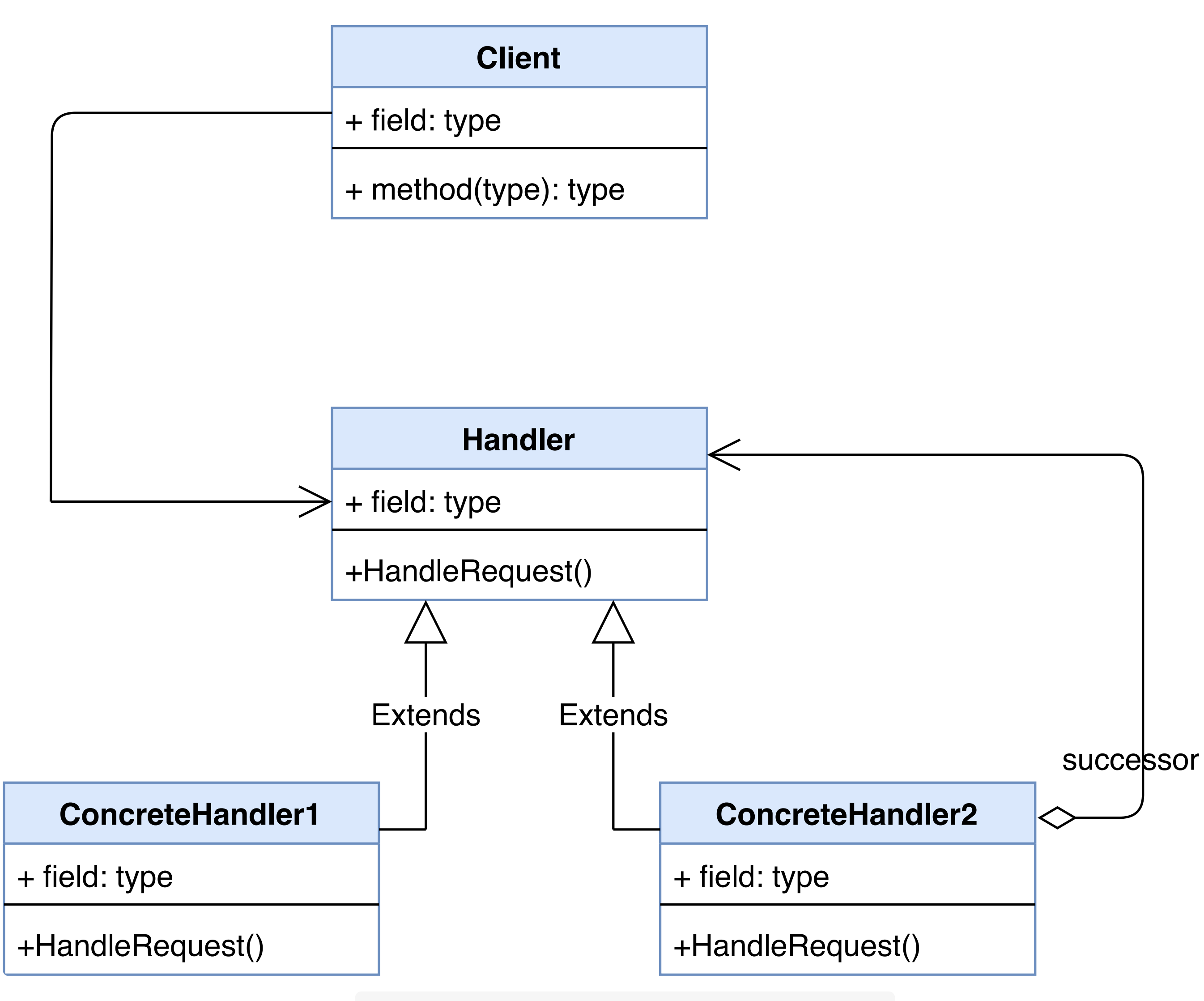
- **Concrete Handlers**:

  These are the classes that implement the Handler interface. Each concrete handler is responsible for some specific function or task. If a handler cannot handle the incoming request, it is forwarded to the next handler in the chain.

- **Client**:

  This class of the application initiates the request. The Client class may compose chains of handlers dynamically, depending upon its requirements.

In real practice, the `Client` sends its request to the first `ConcreteHandler`. If it can handle the incoming request, it does so; otherwise, forward it to the next handler in the chain. This process continues till the request is handled or it reaches the end of the chain.



Chain of Responsibility Pattern - Class Diagram

Implementation of Chain of Responsibility Pattern

Reconsider the customer support system example provided in the previous section. In this system, different types of queries (like Technical, Billing, and General) are handled by respective departments.

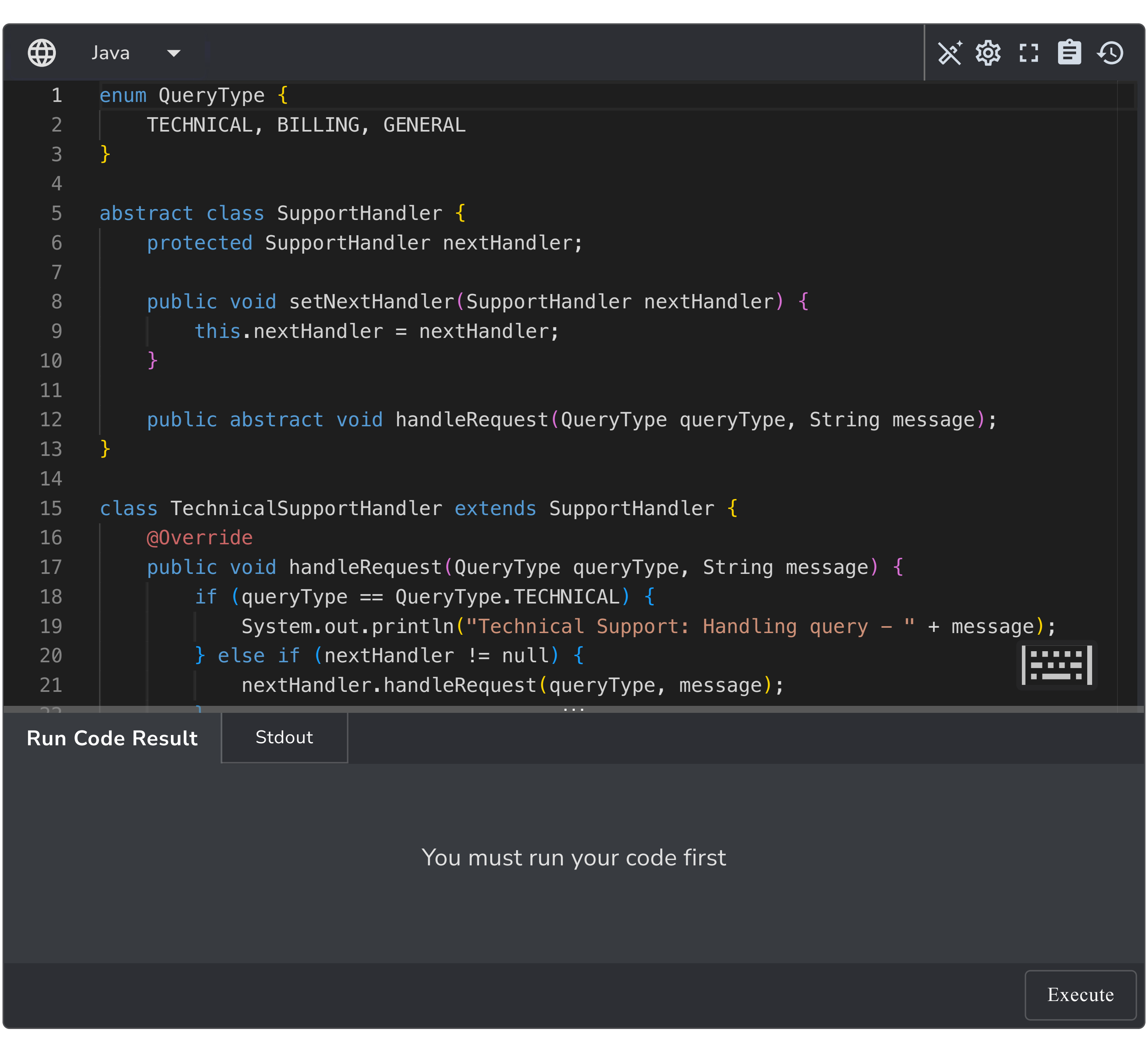
Let's have a look at the pseudocode of this system:

```
// Handler Interface
abstract class SupportHandler {
    protected SupportHandler nextHandler;

    function setNextHandler(nextHandler: SupportHandler) {
        this.nextHandler = nextHandler;
    }

    abstract function handleRequest(queryType: QueryType, message: String);
}

// Concrete Handlers
class TechnicalSupportHandler extends SupportHandler {
    function handleRequest(queryType: QueryType, message: String) {
        if (queryType == TECHNICAL) {
            print("Technical Support: Handling query - " + message);
        } else if (nextHandler != null) {
            nextHandler.handleRequest(queryType, message);
        }
    }
}

class BillingSupportHandler extends SupportHandler {
    function handleRequest(queryType: QueryType, message: String) {
        if (queryType == BILLING) {
            print("Billing Support: Handling query - " + message);
        } else if (nextHandler != null) {
            nextHandler.handleRequest(queryType, message);
        }
    }
}

class GeneralSupportHandler extends SupportHandler {
    function handleRequest(queryType: QueryType, message: String) {
        if (queryType == GENERAL) {
            print("General Support: Handling query - " + message);
        } else if (nextHandler != null) {
            nextHandler.handleRequest(queryType, message);
        }
    }
}

// Client
class SupportChain {
    static function getSupportChain(): SupportHandler {
        technicalSupport = new TechnicalSupportHandler();
        billingSupport = new BillingSupportHandler();
        generalSupport = new GeneralSupportHandler();

        technicalSupport.setNextHandler(billingSupport);
        billingSupport.setNextHandler(generalSupport);

        return technicalSupport;
    }
}

// Usage
supportChain = SupportChain.getSupportChain();
supportChain.handleRequest(TECHNICAL, "I can't connect to the internet.");
supportChain.handleRequest(BILLING, "I have a question about my invoice.");
supportChain.handleRequest(GENERAL, "Thank you for your service.");
```

- **SupportHandler**: This is an abstract class that defines a structure for handling requests. It has a method `handleRequest()` and a function `setNextHandler()` to create a link to the next handler in the chain.
- **Concrete Handlers**: `BillingSupportHander`, `TechnicalSupportHandler`, and GeneralSupportHandler are the specific handlers that handle the different types of queries.
- **SupportChain**: This class assembles the chain of all the support handlers.
- **Client**: The client class (or main application) initiates the support chain and sends specific queries. The queries are then handled by the appropriate handler in the chain.

Using this design pattern, all the queries are managed properly by the specific handler. This creates the flexibility to add or rearrange handlers without disturbing the previous handlers in the system

Implementation



```java
enum QueryType {
    TECHNICAL, BILLING, GENERAL
}

abstract class SupportHandler {
    protected SupportHandler nextHandler;

    public void setNextHandler(SupportHandler nextHandler) {
        this.nextHandler = nextHandler;
    }

    public abstract void handleRequest(QueryType queryType, String message);
}

class TechnicalSupportHandler extends SupportHandler {
    @Override
    public void handleRequest(QueryType queryType, String message) {
        if (queryType == QueryType.TECHNICAL) {
            System.out.println("Technical Support: Handling query - " + message);
        } else if (nextHandler != null) {
            nextHandler.handleRequest(queryType, message);
```

You must run your code first

Application of Chain of Responsibility Pattern

- **Graphical User Interface (GUI) Event Handling**: This pattern is commonly used in GUI frameworks that allow an event to be passed through a series of elements. When a user clicks a button, for example, the event may be recorded by the button itself, the containing panel, and the main application window in order of precedence, allowing various elements to react or ignore the event.

- **Middleware in Web Applications**: This pattern is used by middleware components in web servers or frameworks to process HTTP requests. Every middleware component has the ability to process, alter, and decide whether to forward the request to the subsequent middleware in the stack or to break the chain if it has been fully processed.

- **Logging Systems**: Different loggers can be configured to handle messages of varying severity. For instance, within a single chain, a console logger can handle debug logs, error logs can be sent to a file, and critical logs can be emailed to administrators.

- **Approvals in Business Processes**: Within organizational workflows, particularly in approval processes, a request, such as a report on expenses or a request for a leave of absence, may be referred to a series of authorities (manager, HR, supervisor, etc.), each of whom has the power to either approve or refer the request to a higher level.

These examples show how the Chain of Responsibility pattern facilitates better separation of concerns in complex procedures and helps design systems that are simple to expand and maintain.

Pros and Cons

The Chain of Responsibility design pattern offers several advantages but also comes with certain drawbacks. Here are its pros and cons:

Pros	Cons
Decoupling Sender and Receiver It promotes loose coupling by separating request initiators and handlers.	**Risk of Unhandled Requests** There's a possibility that a request may go unhandled if the chain doesn't include an appropriate handler.
Flexibility in Responsibilities Enables the user to add or remove handlers dynamically.	**Performance Overhead** Passing requests through the chain can effect performance, especially if there are long chains.
Increased Extensibility New handlers can be easily added to handle new types of requests.	**Debugging Difficulty** If there are errors, debugging through the chain can be challenging.
Separation of Concerns Each handler has its own specific responsibility that leads to cleaner and manageable code.	**Complex Chain Configuration** Setting up and maintaining the chain can be complex in large or dynamic systems.
Simplified Object Interfaces Handlers don't need knowledge of each other's interfaces, simplifying interactions.	

In summary, the Chain of Responsibility pattern is a powerful tool for handling requests in a flexible and dynamic way, adaptable to a range of scenarios while maintaining code simplicity and extendibility.

← Previous
Introduction
Next →
Command Pattern
☐ Mark as Completed